

Hive vs RDBMS (MySQL) — Core Differences & Exceptions

Purpose & Architecture

Concept	MySQL	Hive
Type	Online Transaction Processing (OLTP)	Online Analytical Processing (OLAP)
Goal	Fast reads/writes for small records (banking, web apps)	Fast reads for huge datasets (TB–PB scale)
Storage	Row-based (InnoDB/MyISAM)	File-based (HDFS, ORC, Parquet)
Execution Engine	SQL engine in memory	Converts SQL → MapReduce / Tez /

So, Hive ≠ Database; it's a SQL-like layer over Hadoop.

Data Insertion & Updates

Feature	MySQL	Hive
INSERT	Row-by-row (fast, transactional)	Supported but slow (batch-oriented)
UPDATE	Yes (row-level)	Not natively supported (only in ACID tables)

DELETE	Yes	Not supported unless using ACID-managed tables
TRUNCATE	Yes	Works
MERGE	Yes	Works only for transactional tables

Hive ACID tables (since Hive 0.14+) allow UPDATE, DELETE, but only if:

- Table is STORED AS ORC
 - TBLPROPERTIES('transactional'='true')
- Otherwise, Hive is append-only.

Schema & Metadata

Concept	MySQL	Hive
Schema enforcement	Strict (schema on write)	Lazy (schema on read)
Null handling	Default value if not provided	Reads whatever text file has
Constraints	Primary key, foreign key, unique, not null	Only informational (not enforced)

Hive *stores metadata* in the Hive Metastore (often backed by MySQL), but it does not enforce data integrity.

So, even if you declare:

```
CREATE TABLE t1(id INT PRIMARY KEY);
```

Hive won't stop duplicate IDs — it's just for documentation.

Transactions & Concurrency

Concept	MySQL	Hive
Transactions (ACID)	Fully supported	Only for ORC + transactional tables
Locking	Row-level	Table-level (coarse)
Real-time updates	Yes	No (batch-based)

Hive is designed for batch analytics, not real-time updates.

5 Query Execution

Concept	MySQL	Hive
SQL execution	In-memory	Generates MapReduce / Tez / Spark jobs
Execution time	Milliseconds	Seconds to minutes
Indexes	B-trees, hash indexes	Limited (bitmap indexing only in ORC)
Joins	In-memory joins	Distributed joins (expensive unless optimized)

Hive optimizations:

- Partition pruning
- Bucketing
- Predicate pushdown (in ORC)
- Vectorized execution

But still slower for small queries — Hive is for big data, not microqueries.

Data Types & Functions

Concept	MySQL	Hive
Date/Time	Native DATE, DATETIME	Has DATE, TIMESTAMP, but limited manipulation
String functions	Full (REPLACE, SUBSTRING, CONCAT, etc.)	Many supported, but syntax differs
Aggregates	Same (SUM, AVG, etc.)	Same
Subqueries	Fully supported	Supported (but not always efficient)
Window functions	Yes	(since Hive 0.11+)

So — functionality similar, but slower due to MapReduce overhead.

Joins

Join type	MySQL	Hive
Inner, left, right, full	Yes	Yes
Cross join	Yes	(but must enable <code>hive.strict.checks.cartesian.product=false</code>)
Distributed joins	No	yes (map-side, sort-merge, shuffle joins)

Storage & Data Access

Concept	MySQL	Hive
Storage location	Inside database	In HDFS (external)
Compression	InnoDB-level	ORC/Parquet with built-in compression
Access	JDBC/ODBC	HiveServer2, Beeline, JDBC
File format	Proprietary	Text, ORC, Parquet, Avro, JSON, etc.

Hive = schema + SQL interface over files,

MySQL = real database engine managing pages, locks, and indexes.

Functions that behave differently or fail

Function / Feature	MySQL	Hive Behavior
AUTO_INCREMENT	Works	Not supported
LIMIT	Works fast	Works, but slow (reads full file first)
NOW() / SYSDATE()	Works	Works
INSERT ... SELECT	Works	Works, but batch job (slow)
ORDER BY	Works fully	Works, but sorts entire dataset with 1 reducer (slow)
DISTRIBUTE BY / SORT BY	No	Hive-specific, for distributed sorting
PRIMARY KEY / FOREIGN KEY	Enforced	Metadata only
Transactions	Real	Only for ACID ORC tables

Analytical Strengths (where Hive wins)

Handles petabytes of data

Runs on distributed clusters

Integrates with Spark, Tez, HDFS, YARN

Great for batch ETL, reporting, data warehousing

But not suitable for:

- **Real-time applications**
- **Rapid OLTP transactions**

- High-concurrency small queries

Summary

Category	MySQL	Hive
Type	OLTP	OLAP
Storage	Local DB	HDFS
Real-time CRUD	yes	No
Schema Enforcement	Strict	Loose
Joins	In-memory	Distributed
Indexes	Rich	Minimal
Transactions	Full	Limited
Query speed (small data)	Fast	Slow
Query speed (big data)	Limited	Scalable
Data size support	GBs	TB–PB
Ideal for	Web apps	Data warehousing

Hive practical :

hive

hive > set hive.cli.print.current.db = true;

hive(default)>

create database tan;

folder - training.db will get created under /user/hive/warehouse

show databases;

use tan;

create table nyse (

stk_date date,

symbol string,

Open double,

Close double,

low double,

High double,

Volume double)

ROW FORMAT DELIMITED

FIELDS TERMINATED BY ','

STORED AS TEXTFILE

LOCATION '/user/cdacoctnov012/db';

on different terminal :

hdfs dfs -mkdir db

hdfs dfs -put nyse.csv db/

Found 1 items

-rw-r--r-- 3 cdacoctnov012 cdacoctnov012 51667341 2025-11-09 10:03 db/nyse.csv

Load data inpath '/user/cdacoctnov012/db/nyse.csv' into table nyse;

Check this again hdfs dfs -ls db

the data is transferred to hive data ware house

```
> select * from nyse limit 10;
```

OK

```
NULL  symbol NULL  NULL  NULL  NULL  NULL
2016-01-05  WLTW  123.43 125.839996  122.309998  126.25 2163600.0
2016-01-06  WLTW  125.239998  119.980003  119.940002  125.540001
2386400.0
```

When Hive loads the file, it treats every line as a data row — including this header line.

So Hive tries to convert "stk_date" (a string) into a DATE, "Open" into a DOUBLE, etc., which all become NULL (since conversion fails).

```
ALTER TABLE nyse SET TBLPROPERTIES ("skip.header.line.count"="1");
```

this can also be used during table creation as follows:

```
CREATE TABLE nyse (
    stk_date DATE,
    symbol STRING,
    open DOUBLE,
    close DOUBLE,
    low DOUBLE,
    high DOUBLE,
    volume DOUBLE
)
```

ROW FORMAT DELIMITED

FIELDS TERMINATED BY ','

STORED AS TEXTFILE

TBLPROPERTIES ("skip.header.line.count"="1");

How to check the header now ?

set hive.cli.print.header =true;

fetch hive tables from cli:

hive -e "SELECT * FROM nyse LIMIT 5;"

hive -e "set hive.cli.print.header=true; SELECT * FROM nyse LIMIT 10;" | column -t

Q1. What is Hive and how is it different from a traditional RDBMS?

Answer:

Hive is a data warehouse infrastructure built on top of Hadoop that allows querying large datasets stored in HDFS using SQL-like syntax (HiveQL).

Unlike RDBMS:

- Hive queries are converted into MapReduce (or Tez/Spark) jobs.
 - Hive is designed for batch processing, not for real-time queries.
 - Hive is schema-on-read (applies schema while reading data).
-

Q2. Create a table from a CSV file stored in HDFS.

Answer:

```
CREATE TABLE nyse (  
    stk_date STRING,  
    symbol STRING,  
    open FLOAT,  
    close FLOAT,  
    low FLOAT,  
    high FLOAT,  
    volume FLOAT)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
STORED AS TEXTFILE  
LOCATION 'hdfs://master:9000/user/hive/warehouse/nishad_one.db/nyse'  
TBLPROPERTIES ("skip.header.line.count"="1");
```

```
hadoop fs -ls hdfs://master:9000/user/hive/warehouse/nishad_one.db/
```

```
hadoop fs -mkdir hdfs://master:9000/user/hive/warehouse/nishad_one.db/nyse
```

```
hadoop fs -put nyse.csv  
hdfs://master:9000/user/hive/warehouse/nishad_one.db/nyse/
```

```
cp /home/cdacoctnov012/nyse.csv /home/cdacoctnov005/
```

Q3. View only the schema of a Hive table.

Answer:

```
DESCRIBE nyse;
```

Q4. Display the first 10 records.

Answer:

```
SELECT * FROM nyse LIMIT 10;
```

Q5. Count how many rows are in the table.

Answer:

```
SELECT COUNT(*) FROM nyse;
```

Q6. Find the average closing price per symbol.

Answer:

```
SELECT symbol, AVG(close) AS avg_close  
FROM nyse  
GROUP BY symbol;
```

Q7. Find the highest traded volume for each stock symbol.

Answer:

```
SELECT symbol, MAX(volume) AS max_volume  
FROM nyse  
GROUP BY symbol;
```

What Is a Partition in Hive?

A partition in Hive means dividing a table into smaller, more manageable pieces based on the values of one or more columns — typically columns that are used often in **WHERE** clauses.

Think of it like having folders inside folders.

```
/user/hive/warehouse/nyse_partitioned/year=2015/  
/user/hive/warehouse/nyse_partitioned/year=2016/  
/user/hive/warehouse/nyse_partitioned/year=2017/
```

Q8. Create a partitioned table by year.

Answer:

```
CREATE TABLE nyse_partitioned (  
    stk_date STRING,  
    symbol STRING,  
    open FLOAT,  
    close FLOAT,  
    low FLOAT,  
    high FLOAT,  
    volume FLOAT  
)  
PARTITIONED BY (year STRING)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY '\t';  
-----
```

PARTITIONED BY (year STRING)

This tells Hive that the data will be divided (partitioned) by the year column.

The column year is not part of the main data — it's used as a directory name in HDFS.

ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t'

The data fields in the file are separated by tabs (\t).

Because i dont have different csv for each year

Instead, we'll let **Hive create partitions automatically** using **dynamic partitioning**.

Or else you have a direct file static partition can be done

```
–LOAD DATA INPATH 'hdfs://master:9000/user/data/nyse_2016.csv' INTO TABLE  
nyse_partitioned PARTITION (year='2016'); // same command for each year
```

```
SET hive.exec.dynamic.partition=true;  
SET hive.exec.dynamic.partition.mode=nonstrict;
```

To load data:

```
INSERT OVERWRITE TABLE nyse_partitioned  
PARTITION (year)  
SELECT  
    stk_date,  
    symbol,  
    open,  
    close,  
    low,  
    high,  
    volume,  
    year(stk_date) AS year  
FROM nyse;
```

Time comparison :

A. On the non-partitioned table (nyse)

```
SELECT COUNT(*)  
FROM nyse  
WHERE year(stk_date) = 2016;
```

```
SELECT symbol, AVG(close) AS avg_close_2016 FROM nyse WHERE year(stk_date) = 2016  
GROUP BY symbol;
```

B. On the partitioned table (nyse_partitioned)

```
SELECT COUNT(*)  
FROM nyse_partitioned  
WHERE year = '2016';
```

```
SELECT symbol, AVG(close) AS avg_close_2016 FROM nyse_partitioned WHERE  
year(stk_date) = 2016 GROUP BY symbol;
```

Q9. Show all partitions in a table.

Answer:

```
SHOW PARTITIONS nyse_partitioned;
```

How Partitioning Helps

Feature	Explanation
Improves Query Performance	Hive only scans relevant partitions (folders), not the entire dataset. Example: If you query only year= ' 2016 ' , it skips other years.
Reduces Data Scanning Cost	Less data read from HDFS = faster query and lower resource usage.
Logical Data Organization	Easier to manage large datasets (e.g., by year, month, region, etc.).
Supports Incremental Loads	You can add data for a new partition (say, year 2025) without touching old data.

Q10. Find stocks with closing price greater than 120.

Answer:

```
SELECT * FROM nyse WHERE close > 120;
```

Q11. Display top 5 records by highest volume.

Answer:

```
SELECT * FROM nyse ORDER BY volume DESC LIMIT 5;
```

What is Bucketing in Hive?

Bucketing is a way to further divide data inside each partition (or table) into smaller, fixed-size files called buckets — based on the hash value of a column.

Think of it like this:

Partitioning splits data by a *category* (e.g., year = 2016, 2017...)

Bucketing splits data *within each partition* into equal-sized buckets based on a *column's hash value* (e.g., by **symbol).**

Q12. Create a bucketed table with 4 buckets.

Answer:

```
CREATE TABLE nyse_bucketed (  
    stk_date STRING,  
    symbol STRING,  
    open FLOAT,  
    close FLOAT,  
    low FLOAT,  
    high FLOAT,  
    volume FLOAT  
)  
CLUSTERED BY (symbol) INTO 4 BUCKETS  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY '\t';
```

How Hive Decides the Bucket

Hive computes:

```
bucket_number = hash(symbol) % number_of_buckets
```

So if symbol= ' AAPL ' hashes to 2, it goes into bucket 2.

Thus, data with the **same symbol value always goes to the same bucket** — even if you reprocess later (for join optimization).

example

symbol	Java hashCode	After mod 4	Goes to bucket
AAPL	63281924	0	bucket_0
MSFT	77464855	3	bucket_3
WLTW	88742467	3	bucket_3
TATA	2656907	3	bucket_3
INFY	23456781	1	bucket_1

/nyse_bucketed/000000_0 → contains AAPL

/nyse_bucketed/000001_0 → contains INFY

/nyse_bucketed/000003_0 → contains MSFT, WLTW, TATA

Loading Data into a Bucketed Table:

```
SET hive.enforce.bucketing=true;
```

```
INSERT OVERWRITE TABLE nyse_bucketed SELECT * FROM nyse;
```

Difference Between Partitioning and Bucketing

Feature	Partitioning	Bucketing
Basis	Column values (e.g., year=2016)	Hash of a column (e.g., hash(symbol) % 4)
Data location	Different folders per partition	Different files within same folder
Granularity	Coarse (macro-level split)	Fine (micro-level split)
Used for	Query pruning	Faster joins, balanced data
Example	PARTITIONED BY (year STRING)	CLUSTERED BY (symbol) INTO 4 BUCKETS

Q18. Create a managed table from an existing table.

Answer:

```
CREATE TABLE nyse_copy AS  
SELECT * FROM nyse;
```

Q19. Export Hive table data to HDFS.

Answer:

```
INSERT OVERWRITE DIRECTORY '/user/cdacoctnov012/db'  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY '\t'  
SELECT * FROM nyse;
```

Q20. Drop a Hive table (and delete data).

Answer:

DROP TABLE nyse;

External